

自觉遵守考场纪律
如考试作弊
此答卷无效

姓名

学号

东南大学 考试卷 (A 卷)

课 程 名 称	数据结构	考 试 学 期	10-11-3	得 分
适 用 专 业	吴健雄学院 610	考 试 形 式	闭卷	考试时长 120 分钟

一、选择题 (每题 1 分, 共 5 分)

1. 设有一个二维数组 $A[m][n]$, 如果 $A[0][0]$ 的首地址为 644(10 进制), $A[2][2]$ 的首地址为 676, 每个元素占一个字节, 则 $A[4][5]$ 的首地址为 ()。
- A. 692 B. 626 C. 709 D. 724
2. 若让元素 1, 2, 3 依次但并非连续进栈, 则哪种出栈次序是不可能的 ()
- A. 3, 2, 1 B. 2, 1, 3
C. 3, 1, 2 D. 1, 2, 3
3. 设完全二叉树有 82 个结点, 从根结点开始顺序编号, 根节点为 1 号, 其他结点自上向下, 同一层次自左向右连续编号。则第 41 号结点的双亲结点的编号为 ()
- A. 20 B. 21 C. 81 D. 82
4. 采用对半搜索算法搜索长度为 n 的有序表, 元素的平均搜索长度为 ()
- A. $O(n^2)$ B. $O(n)$ C. $O(n \log_2 n)$ D. $O(\log_2 n)$
5. 采用邻接表存储的图的深度优先遍历算法类似于二叉树的 ()
- A. 中序遍历 B. 前序遍历 C. 后序遍历 D. 按层次遍历

二、判断题 (每题 1 分, 共 5 分)

1. 邻接表只能用于有向图的存储, 邻接矩阵对于有向图和无向图的存储都适用。 ()
2. 直接选择排序是一种不稳定的排序方法。 ()
3. 在用散列表存储关键码集合时, 可以用双散列法寻找下一个空桶。在设计再散列函数时, 要求计算出的值与表的大小 m 互质。 ()
4. 连通分量是无向图中的极小连通子图。 ()
5. 若有一个叶子节点是二叉树中某子树的前序遍历结果序列的最后一个结点, 它一定是该子树的中序遍历结果序列的最后一个结点。 ()

三、填空题（每空 1 分， 10 分）

1. 每次从表的无序部分取出一个元素，把它插入到表的有序部分的适当位置，此种排序方法叫作 (1) 排序；每次从表的无序部分中挑选出一个最小或最大元素，把它与表的有序部分的最后一元素交换，此种排序方法叫作 (2) 排序。
2. 中缀表达式 “ $a*b/(x+2)+25$ ” 所对应的后缀表达式为 (3)
3. 后缀表达式 “ $ab/c-de*+ac*-$ ” 所对应的中缀表达式为 (4)
4. 对于一个具有 n 个顶点和 e 条边的有向图和无向图，在其对应的邻接表中，所含边结点数分别为 (5) 和 (6) 条。
5. 当向最小堆插入一个新元素时，应该首先成为堆的 (7) 元素，然后逐层 (8) 调整，直到调整到适当位置。 当从一个最小堆删除一个元素时，需要把 (9) 元素填补到堆顶位置，然后逐层 (10) 调整，直到调整到适当位置。

四、简答简述题（每题 8 分，共 24 分）

1. 已知一棵树的广义表表示为： $A(B(E(K,L),F),C(G(M,N)),D(H(O),I,J))$ 请绘出该树的示意图，并画出其链式结构图。

示意图：

链式结构图：

2. 待排序序列有 6 个元素 [21,25,49,25*,16,08] , 以第 1 个元素 21 作为基准开始进行快速排序。首先绘出第 1 趟排序的详细过程, 然后绘出各趟排序的结果。

第 1 趟排序的详细过程：

各趟排序的结果（只需给出每一趟排序开始状态和每一趟排序后的结果）：

3. 根据下列条件绘出学生必修课程的 AOV 网络图、采用邻接表表示并给出入度表。建立入度为 0 的顶点栈 (不一定借用入度表)及其在建立 AOV 网络拓扑排序过程中的变化。用图解法一步一步完成拓扑排序。最后给出课程学习次序的拓扑有序 AOV 网络。

课程代号	课程名称	先修课程
C1	高等数学	
C2	程序设计基础	
C3	离散数学	C1、 C2
C4	数据结构	C3、 C2
C5	高级语言程序设计	C2
C6	编译方法	C5、 C4
C7	操作系统	C4、 C9
C8	普通物理	C1
C9	计算机原理	C8

五、综合算法题（每空 2 分，共 56 分）

1. 以下是最大堆的定义、插入与删除操作，请完善以下各程序段。

```
template<typename T> class Maxheap: public MaxPQ<T>{
    Element<T>* heap;
    int n;
    int MaxSize;
public :
    Maxheap(int sz=Defaultsize); //创建空堆，最多可以容纳    sz个元素
    void Insert(Element<T>& item);
    Element<T>* Delete(Element<T>& x);
    void show();
};
template<typename T>Maxheap<T>::Maxheap( int sz){
    MaxSize=sz;
    n=0;

    heap=(1)_____ ; //建立堆， heap[0]不用
}
template<typename T> void Maxheap<T>::Insert(Element<T>& x){
    int i;
    if(n==MaxSize){
        cerr<<"heap is full.\n";
        return ;
    }
    n++;
    for(i=n;i>1;){
        //i==1 表示已达到根节点

        if (x<=(2)_____) break;//新元素的关键字不大于结点    i的双亲的关键字

        (3)_____;//heap[i] 未占用，将双亲结点元素移到结点    i中

        (4)_____ ; //继续向上
    }
    (5)_____ ; //x 的位置定了，再放数值进去
}
template<typename T>Element<T>* Maxheap<T>::Delete(Element<T>& x){
    int i,j;
    if(!n){
        cerr<<"heap is empty.\n";
        return NULL;
    }
    x=heap[1];
```

```

Element<T> k=heap[n]; //候补的结点为最大下标元素，原来应该将它放在堆顶，
                        //为了简化算法，暂时放在工作对象    k中
n--;                    //堆元素数减 1
for(i=1,j=2;j<=n;){      //j是i的子女

    if (j<n) if (heap[j]<heap[j+1]) (6)_____ ; //j指向较大子女

    if ((7)_____) break; //候补的结点大，不再移动

    (8)_____ ;//还需移动，因 heap[i] 未占用，将较大子女移入

    (9)_____ ;//为下一趟移动，改变循环变量    i

    (10)_____ ; //为下一趟移动，改变循环变量    j

}

(11)_____ ; //候补的结点放在正确的位置上
return &x;
}

```

2. 请完善二叉树利用栈的前序遍历和中序遍历非递归算法。

```

template <typename T>void BinaryTree<T>::PreOrder1(void (*visit) (BinTreeNode<T> *t) ) {
//非递归前序遍历
    LinkStack<BinTreeNode<T>*> S;
    BinTreeNode<T> *p = root;
    S.Push (NULL);
    while (p != NULL) {
        visit(p);                //访问结点
        if (p->rightChild != NULL)
            (12)_____ ;        //预留右指针在栈中
        if (p->leftChild != NULL)
            (13)_____ ;        //进左子树
        (14)_____ ;            //左子树为空，由堆栈弹出
    }
}
}

```

```

template <typename T> void BinaryTree<T>::InOrder1(void (*visit) (BinTreeNode<T> *t)) {
//非递归中序遍历
    LinkStack<BinTreeNode<T>*> S;
    BinTreeNode<T> *p = root;
    do {

```

```

while (p != NULL) {
    (15)_____ ;           //该子树沿途结点进栈
    (16)_____ ;           //遍历指针向左下移动
}
if (!S.IsEmpty()) {        //栈不空时退栈
    (17)_____ ; visit (p); //退栈，访问
    (18)_____ ;           //遍历指针进到右子女
}
} while (p != NULL || !S.IsEmpty ());
}

```

3. 完善 / 邻接矩阵存储的图的类定义（无向图）的一些成员函数

```

template <class T, class E>class Graphmtx : public Graph<T,E>{           //图的邻接矩阵类定义
public:
    Graphmtx(int sz = maxSize);                                           //构造函数
    ~Graphmtx(){                                                           //析构函数
        delete []VertexList;
        delete []Edge;
    }
    T getValue(int i){                                                     //取顶点 i 的值, i 不合理返回 0
        return i >= 0 && i < numVertices ? VertexList[i] : 0;
    }
    E getWeight(int v1, int v2){                                           //取边 (v1,v2) 上的权值, 若边不合理, 则返回 0
        return v1 != -1 && v2 != -1 ? Edge[v1][v2] : 0;
    }
    int getFirstNeighbor(int v);                                           //取顶点 v 的第一个邻接顶点
                                                                    //返回该邻接顶点的编号, 若不存在则返回 -1
    int getNextNeighbor(int v, int w);                                     //取 v 的邻接顶点 w 的下一邻接顶点
                                                                    //返回下一个邻接顶点的编号, 若不存在或参数不合理则返回 -1
    bool insertVertex(const T &vertex);                                    //插入顶点 vertex
                                                                    //接受一个参数, 表示插入顶点的值, 返回 true 表示插入成功
    bool insertEdge(int v1, int v2, E cost); // 插入边 (v1, v2), 权值为 cost, 返 true 表示插入成功
    bool removeVertex(int v); // 删去顶点 v 和所有与它相关联的边, 返回 true 表示删除成功
    bool removeEdge(int v1, int v2); // 在图中删去边 (v1,v2), 返回 true 表示删除成功
    int getVertexPos(const T &vertex){ // 给出顶点 vertex 在图中的位置, 若不存在则返回 -1
        for (int i = 0; i < numVertices; i++){
            if (VertexList[i] == vertex){
                return i;
            }
        }
    }
}

```

```

        }
    }
    return -1;
}
private:
    T *VertexList;           // 顶点表
    E **Edge;                // 邻接矩阵
};
template <class T, class E> Graphmtx<T,E>::Graphmtx(int sz):Graph<T,E>(sz){
    int i, j;
    VertexList = new T[maxVertices];           // 创建顶点表数组
    Edge = (int **) new int *[maxVertices];     // 创建邻接矩阵数组
    for (i = 0; i < maxVertices; i++){
        Edge[i] = new int[maxVertices];
    }
    for (i = 0; i < maxVertices; i++){         // 邻接矩阵初始化
        for (j = 0; j < maxVertices; j++){
            Edge[i][j] = maxWeight;
        }
    }
}
}

```

//给出顶点位置为 v 的第一个邻接顶点的位置 ， 如果找不到 ， 则函数返回 -1。

```

template <class T, class E>
int Graphmtx<T,E>::getFirstNeighbor(int v) {
    if (v != -1){
        for (int col = 0; col < numVertices; col++){
            if ((19)_____){
                return col;
            }
        }
    }
    return -1;
}
}

```

//给出顶点 v 的某邻接顶点 w 的下一个邻接顶点

```

template <class T, class E>
int Graphmtx<T,E>::getNextNeighbor(int v, int w){
    if (v != -1 && w != -1){

```



```

        for (int (20) _____; col < numVertices; col++){
            if (Edge[v][col] > 0 && Edge[v][col] < maxWeight){
                return col;
            }
        }
    }
    return -1;
}

```

//插入顶点 vertex：若顶点表满，则不插入，返回 false；若成功插入，则返回 true。

```

template <class T, class E>bool Graphmtx<T,E>::insertVertex(const T &vertex){
    if (numVertices == maxVertices) {
        return false;
    }
    (21) _____ = vertex;
    return true;
}

```

//插入边 (v1, v2), 权值为 cost

```

template <class T, class E>bool Graphmtx<T,E>::insertEdge(int v1, int v2, E cost){
    if (v1 > -1 && v1 < numVertices && v2 > -1 &&
        v2 < numVertices && Edge[v1][v2] == maxWeight){
        (22) _____ = cost; // 无向图
        numEdges++;
        return true;
    }
    else
    {
        return false;
    }
}

```

//删去顶点 v和所有与它相关联的边

```

template <class T, class E>bool Graphmtx<T,E>::removeVertex(int v){
    if (v < 0 && v >= numVertices){
        //v 不在图中 ,不删除
        return false;
    }
    if (numVertices == 1){
        //只剩一个顶点 ,不删除
        return false;
    }
}

```

```

int i, j;
VertexList[v] =(23)_____ ;    //顶点表中删除该结点
for (i = 0; i < numVertices; i++){          //减去与 v相关联边数
    if ((24)_____){
        numEdges--;
    }
}
for (i = 0; i < numVertices; i++){          //用最后一列填补第 v列
    (25)_____ ;
}
numVertices--;                             //顶点数减一
for (j = 0; j < numVertices; j++){          //用最后一行填补第 v行
    (26)_____ ;
}
return true;
}
//在图中删去边 (v1,v2)
template <class T, class E>bool Graphmtx<T,E>::removeEdge(int v1, int v2){
    if (v1 > -1 && v1 < numVertices && v2 > -1 && v2 < numVertices &&
        Edge[v1][v2] > 0 && Edge[v1][v2] < maxWeight){
        (27)_____ ; //无向图
        (28)_____ ;
        return true;
    }
    else return false;
}
}

```